

Week 6: MAPIE and Creating a GitHub Repo



MAISI



Building Interpretable AI in Healthcare
Michigan Data Science Team - Winter 2026



Week 6 Agenda

01.

Icebreaker!

Every great project work session starts off with a great icebreaker.

02.

Conformal Predictions

How can we investigate the level of confidence our model has in the output it gives?

03.

Git and GitHub

What is version control, why is it important, and how can one set up a repository?

04.

Hands-On Data Science!

Let's use what we've learned to quantify the uncertainty in the models that we have trained!

01 Fun Icebreaker!!

Get to know us and start to learn about
each other with a fun icebreaker!



Icebreaker – Week 6

Share with the people around you!! :)

What is your favorite Olympic sport to watch? (Or, what is your favorite sport?)





02

Conformal Predictions

How can we investigate the level of confidence our model has in the output it gives?



What are Conformal Predictions?

- **Conformal Prediction** generates an interval with guaranteed coverage of a true but unknown parameter of interest.
 - Typically, this is done at the $(1 - \alpha)$ level, where α is the statistical threshold of confidence desired, generally between 0.1 and 0.01.
- Instead of just a point prediction (“This is an apple”), gives a confidence interval (“Our model is 95% confident that this is an apple or a peach.”)
- Assumes that observations are i.i.d. (independent and identically distributed), meaning that their order is *exchangeable* (does not matter)

Why are Conformal Predictions Useful?

- Provides **strict guarantees** on error rates, even for small datasets
 - The interval you are given will certainly contain the true parameter at least $(1 - \alpha)$ percent of the time
- Does NOT assume that the data follow any particular distribution. It also works for both *regression* and *classification* tasks.
- If we get a rather wide interval for our prediction, we know that our model isn't overly sure about which group a certain observation belongs to (in a classification context)

What are the Steps of Conformal Predictions?

- **1. Train:** Split the dataset into a train-calibrate-test split, also, establish your level of confidence you wish to achieve (typically between 90% and 99%)
- **2. Calibrate:** Like Week 4, we utilize probability calibration to make sure that the prevalence of the categories in the model truly reflect their prevalence in the real world.
- **3. Predict:** Use your X_{test} set to make both point predictions (“This is an apple”) and prediction sets (“95% confidence that this is an apple or a peach”)



03

Git and GitHub

What is version control, why is it important, and how can one set up a repository?



What is Git?

- **Git** is a free, open-source distributed version control system (DVCS)
 - Used to track changes in source code during the software development process, particularly when working in large groups or as part of a larger team.
- *Distributed* means that each developer that has access to a certain Git repository has access to the entire history of the program development on their local device.
- Works as something as a time machine, with branches being isolated lines of development (typically just “origin/main”)
 - *Merging* combines these branches into one, using push and pull

What is GitHub?

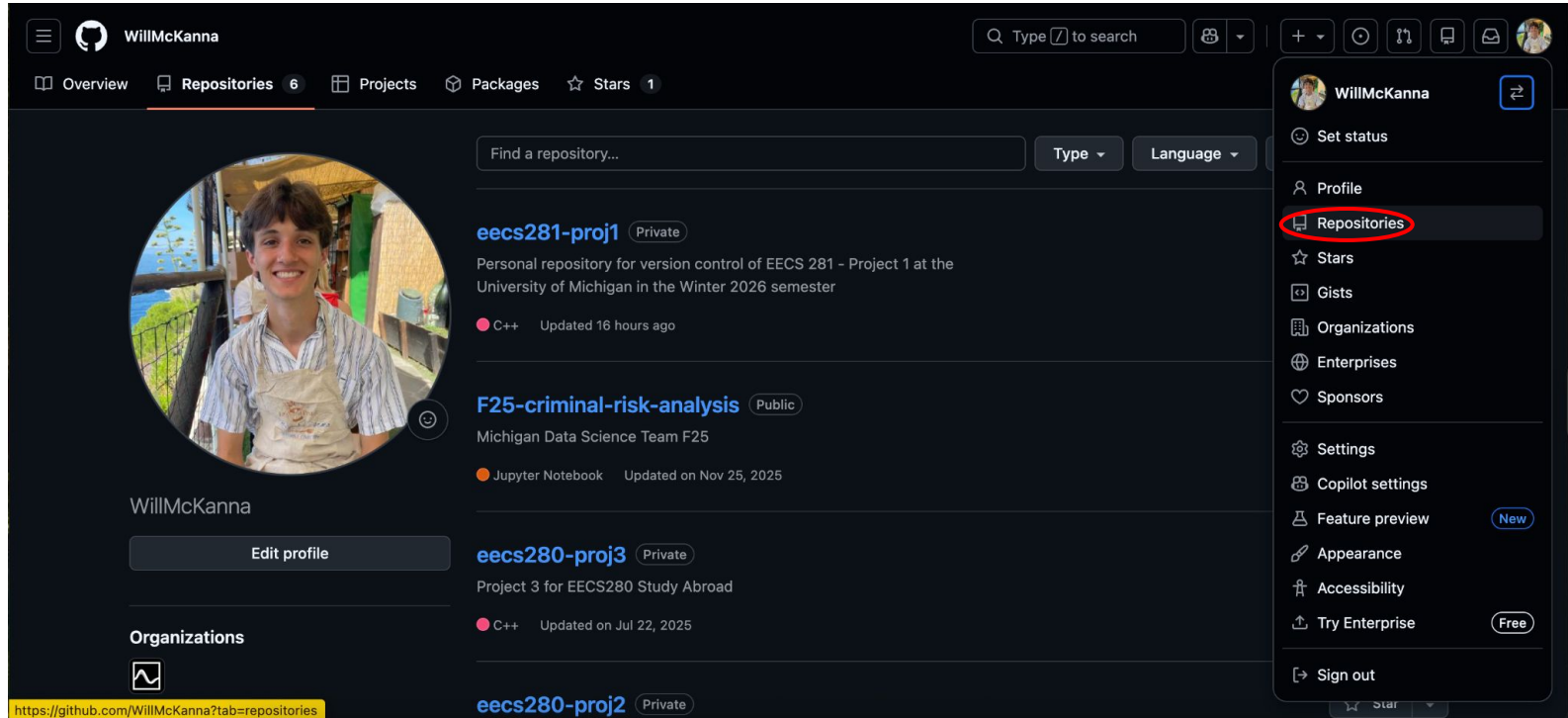
- **GitHub** is a hosting service that affords a navigable interface for Git users to access and download version-controlled software.
- Cloud-based server to host Git content
- Very easy way to track issues, view others' code, and work together with other developers to all work on the same project

	 GitHub
1. It is a software	1. It is a service
2. It is installed locally on the system	2. It is hosted on Web
3. It is a command line tool	3. It provides a graphical interface
4. It is a tool to manage different versions of edits, made to files in a git repository	4. It is a space to upload a copy of the Git repository
5. It provides functionalities like Version Control System Source Code Management	5. It provides functionalities of Git like VCS, Source Code Management as well as adding few of its own features



How to Create a GitHub Repository (yay for version control!)

- I. Navigate to GitHub, click your profile picture, and select “Repositories.” Then, click the green “New” button next to the “Language” dropdown menu



The screenshot shows the GitHub profile page for user WillMcKanna. The 'Repositories' tab is selected in the top navigation bar and highlighted with a red circle in the right-hand sidebar. The profile section on the left includes a circular profile picture of a young man, the name 'WillMcKanna', and an 'Edit profile' button. Below the profile picture, the 'Organizations' section is visible. The main content area displays a list of repositories: 'eecs281-proj1' (Private, C++, updated 16 hours ago), 'F25-criminal-risk-analysis' (Public, Michigan Data Science Team F25, Jupyter Notebook, updated on Nov 25, 2025), 'eecs280-proj3' (Private, C++, updated on Jul 22, 2025), and 'eecs280-proj2' (Private). The right-hand sidebar contains a dropdown menu with options: 'Set status', 'Profile', 'Repositories' (highlighted), 'Stars', 'Gists', 'Organizations', 'Enterprises', 'Sponsors', 'Settings', 'Copilot settings', 'Feature preview' (with a 'New' badge), 'Appearance', 'Accessibility', 'Try Enterprise' (with a 'Free' badge), and 'Sign out'.

WillMcKanna

Overview Repositories 6 Projects Packages Stars 1

Find a repository... Type Language

eecs281-proj1 (Private)
Personal repository for version control of EECS 281 - Project 1 at the University of Michigan in the Winter 2026 semester
C++ Updated 16 hours ago

F25-criminal-risk-analysis (Public)
Michigan Data Science Team F25
Jupyter Notebook Updated on Nov 25, 2025

eecs280-proj3 (Private)
Project 3 for EECS280 Study Abroad
C++ Updated on Jul 22, 2025

eecs280-proj2 (Private)

WillMcKanna

Edit profile

Organizations

WillMcKanna

Set status

Profile

Repositories

Stars

Gists

Organizations

Enterprises

Sponsors

Settings

Copilot settings

Feature preview (New)

Appearance

Accessibility

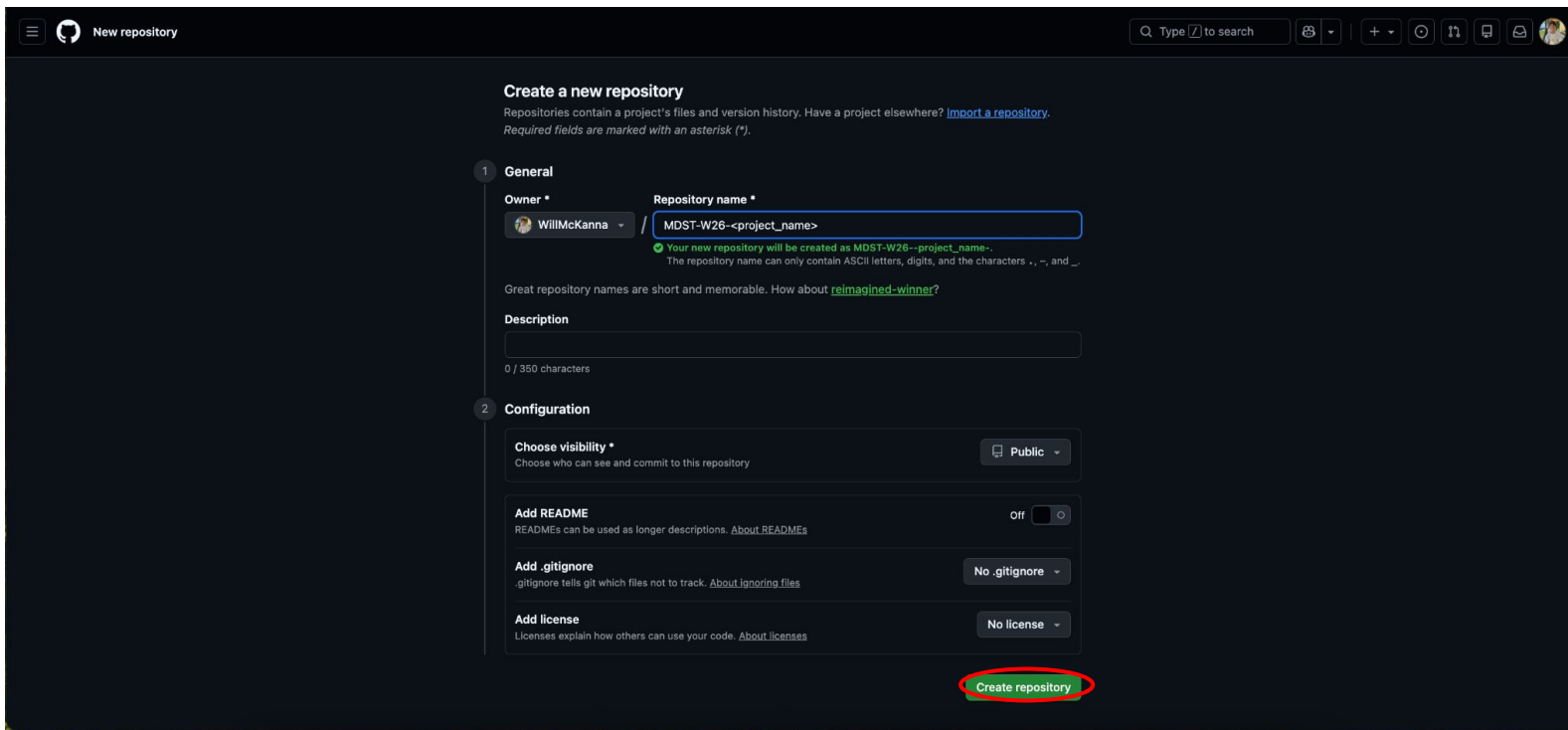
Try Enterprise (Free)

Sign out

<https://github.com/WillMcKanna?tab=repositories>

How to Create a GitHub Repository (yay for version control!)

2. Name your repository something memorable, then click “Create Repository”



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * WillMcKanna / **Repository name *** MDST-W26-<project_name>

✔ Your new repository will be created as MDST-W26-<project_name>-.
The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

Great repository names are short and memorable. How about [reimagined-winner](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#) Off

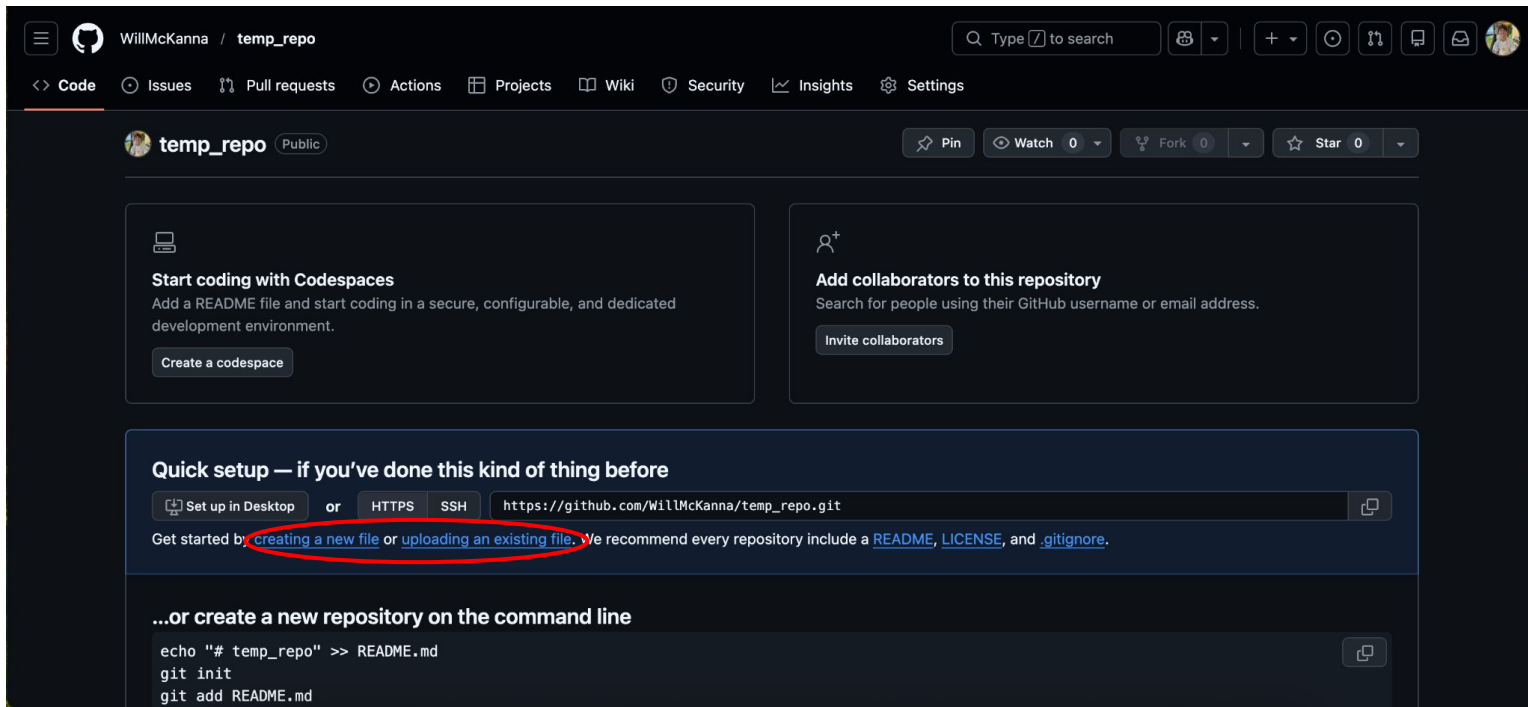
Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#) No license

Create repository

How to Create a GitHub Repository (yay for version control!)

3. Then, populate your repo either with your files, OR click “creating a new file”



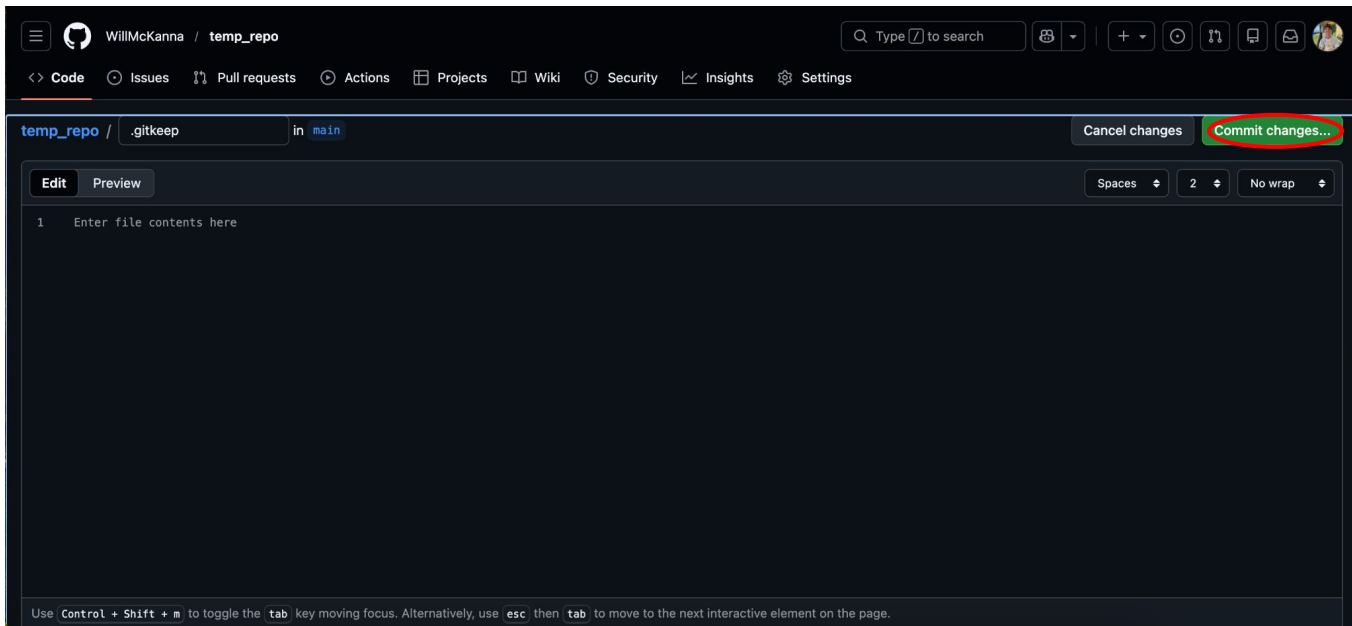
The screenshot shows the GitHub interface for a repository named 'temp_repo' by user 'WillMcKanna'. The repository is public. The page offers several options for getting started:

- Start coding with Codespaces:** A section with a laptop icon and text: "Add a README file and start coding in a secure, configurable, and dedicated development environment." Below this is a button labeled "Create a codespace".
- Add collaborators to this repository:** A section with a person icon and text: "Search for people using their GitHub username or email address." Below this is a button labeled "Invite collaborators".
- Quick setup — if you've done this kind of thing before:** A section with a dark blue background. It shows three options: "Set up in Desktop", "HTTPS", and "SSH". The "SSH" option is selected, and the corresponding URL is displayed: `https://github.com/WillMcKanna/temp_repo.git`. Below this, it says "Get started by creating a new file or uploading an existing file. We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#)." The phrase "creating a new file or uploading an existing file" is circled in red.
- ...or create a new repository on the command line:** A section with a dark blue background showing the following commands:

```
echo "# temp_repo" >> README.md
git init
git add README.md
```

How to Create a GitHub Repository (yay for version control!)

3(b). If you click “create a new file,” entitle it “.gitkeep” and click “Commit changes”



How to Create a GitHub Repository (yay for version control!)

4. Your repo is made! Continue adding to it by clicking the “Add file” dropdown menu

The screenshot shows the GitHub interface for a repository named 'temp_repo' by user 'WillMcKanna'. The repository is public. The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. Below the repository name, there are buttons for Pin, Watch (0), Fork (0), and Star (0). The main content area shows the 'main' branch with 1 branch and 0 tags. A search bar 'Go to file' is present. The 'Add file' dropdown menu is circled in red. Below this, there is a commit history section showing a commit 'a168bb7 · now' by 'WillMcKanna' titled 'Create .gitkeep'. Below the commit history, there is a section for the README file, which is currently empty. The README section has a heading 'Add a README' and a button 'Add a README'. The right sidebar contains sections for 'About' (No description, website, or topics provided), 'Activity' (0 stars, 0 watching, 0 forks), 'Releases' (No releases published, Create a new release), and 'Packages' (No packages published, Publish your first package).



0

4

References

Here are a handful of useful guides to high-utility functions, Git demos, and Colab explanations!

Our Demo! (for reference)



Link:

<https://mdst-ai-in-healthcare.streamlit.app/>

Project Timeline

Week 1 (01/25): Icebreaker/EDA intro, choosing a good dataset

Week 2 (02/01): Data Visualization, Further EDA

Week 3 (02/08): Feature Engineering and Intro to Machine Learning

Week 4 (02/15): Machine Learning and Model Optimization

Week 5 (02/22): Conformal Predictions, Risk Control, and Multi-Class Calibration

NO MEETING 03/01 OR 03/08 – SPRING BREAK

Week 6 (03/15): SHAP Values and Feature Importance Interpretation

Week 7 (03/22): Intro to Streamlit and Project Work Time

Weeks 8 + 9 (03/29 + 04/05): Final Project work time + Data Science Night prep!



Important Functions to Know (pt. 1)

Syntax	Description
<code>import pandas as pd</code>	Import the pandas library, using the alias pd for convenience
<code>import numpy as np</code>	Import the NumPy library, using alias np for numerical operations
<code>import seaborn as sns</code>	Import Seaborn for statistical data visualization
<code>df = pd.read_csv('file.csv')</code>	Load a CSV file into a pandas DataFrame for data manipulation
<code>df.head()</code>	Returns the first 5 rows of the DataFrame, useful for quickly inspecting data
<code>df.info()</code>	Provides a concise summary of the DataFrame, including data types and non-null values

Important Functions to Know (pt. 2)

Syntax	Description
<code>df.describe()</code>	Generates descriptive statistics of numerical columns (mean, median, quartiles, etc.)
<code>df['column_name']</code>	Access a specific column in the DataFrame, works like a key in a dictionary
<code>df.drop(columns=['col'...])</code>	Drops specified columns from the DataFrame, use <code>inplace=True</code> to modify the original DataFrame
<code>df.index</code>	Returns the index labels of the DataFrame
<code>df.isnull()</code>	returns a DataFrame of boolean values , where each entry indicates whether the corresponding value in df is NaN (missing)

Important Functions to Know (pt. 3)

Syntax	Description
<code>df['column'].value_counts()</code>	Returns the count of unique values in a specific column
<code>df['column'].mean()</code>	Returns the mean value of a numerical column
<code>df.corr()</code>	Computes correlation for numerical columns to understand relationships
<code>df.columns</code>	Lists all column names in the DataFrame , useful for renaming or viewing dataset structure
<code>df.shape</code>	Returns # of rows and columns

Important Functions to Know (pt. 4)

Syntax	Description
<code>df.rename(columns={'old' : 'new'}, inplace=True)</code>	Renames specific columns to new names
<code>df.groupby('category')['value']</code>	Groups the DataFrame using a specified column to perform aggregate functions (e.g., <code>.sum()</code> , <code>.mean()</code>)
<code>df['new_column'] = df['column'].apply(function)</code>	Applies a function to each element or column/row
<code>df.shape</code>	Returns a tuple representing the dimensions (rows, columns) of the DataFrame
<code>df.dropna()</code>	Removes rows or columns containing missing values

Important Functions to Know (pt. 5)

Syntax	Description
<code>df.to_numpy()</code>	Converts Pandas DataFrame into a NumPy array
<code>np.shape</code>	Returns a tuple representing the dimensions (rows, columns) of the array (similar to <code>df.shape</code>)
<code>np.reshape()</code>	Reshapes an original numpy array to the specified dimensions (rows, columns)
<code>np.array()</code>	Creates an N-dimensional array (ndarray) which is more memory efficient than standard python lists
<code>np.unique()</code>	Returns the unique elements in a NumPy array (also returns the counts of each element given the keyword argument, <code>return_counts=True</code>)

Important Functions to Know (pt. 6)

Syntax	Description
<code>pd.cut()</code>	Turns continuous numerical data into categorical data
<code>pd.get_dummies(drop_first = True)</code>	Dummy encodes categorical data (used with <code>drop_first</code> argument)
<code>OrdinalEncoder()</code>	Function for encoding categorical information that has an order
<code>train_test_split(X, y, test_size=0.2, random_state=42)</code>	Splits data into a training set and a testing set . The test set size can be changed
<code>LinearRegression()</code>	Used for initializing a linear regression model
<code>LogisticRegression()</code>	Used for initializing a logistic regression model

Important Functions to Know (pt. 7)

Syntax	Description
StandardScaler()	Turns continuous numerical data into categorical data
model.fit(X_train, y_train)	Fits a model to the training data
model.predict(X_test)	Uses the test data to make predictions
.coef_	Attribute for getting the models coefficients (for linear and logistic regression)
mean_squared_error()	Calculates the average squared distance between the predicted and actual data points
mean_absolute_error()	Calculates the average distance between the predicted and actual values

How to Upload Files into Colab

Click on the folder in the sidebar



The screenshot shows a Google Colab notebook interface. On the left, the 'Files' sidebar is open, showing a folder named 'sample_data'. A red arrow points to this folder from the text 'Click on the folder in the sidebar'. The notebook content area has a title 'Week 1 - Pandas Practice'. Below the title, there is a text prompt: 'Here is where you import the libraries necessary to perform the following tasks!'. The first code cell contains the following code:

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

Below the code cell, it says 'Mounted at /content/drive'. The second text prompt says 'Load the Google Forms .csv into a Pandas dataframe.' The second code cell contains:

```
df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

The third text prompt says 'Print out the .head() and the datatypes.' The third code cell contains:

```
df.head()
```

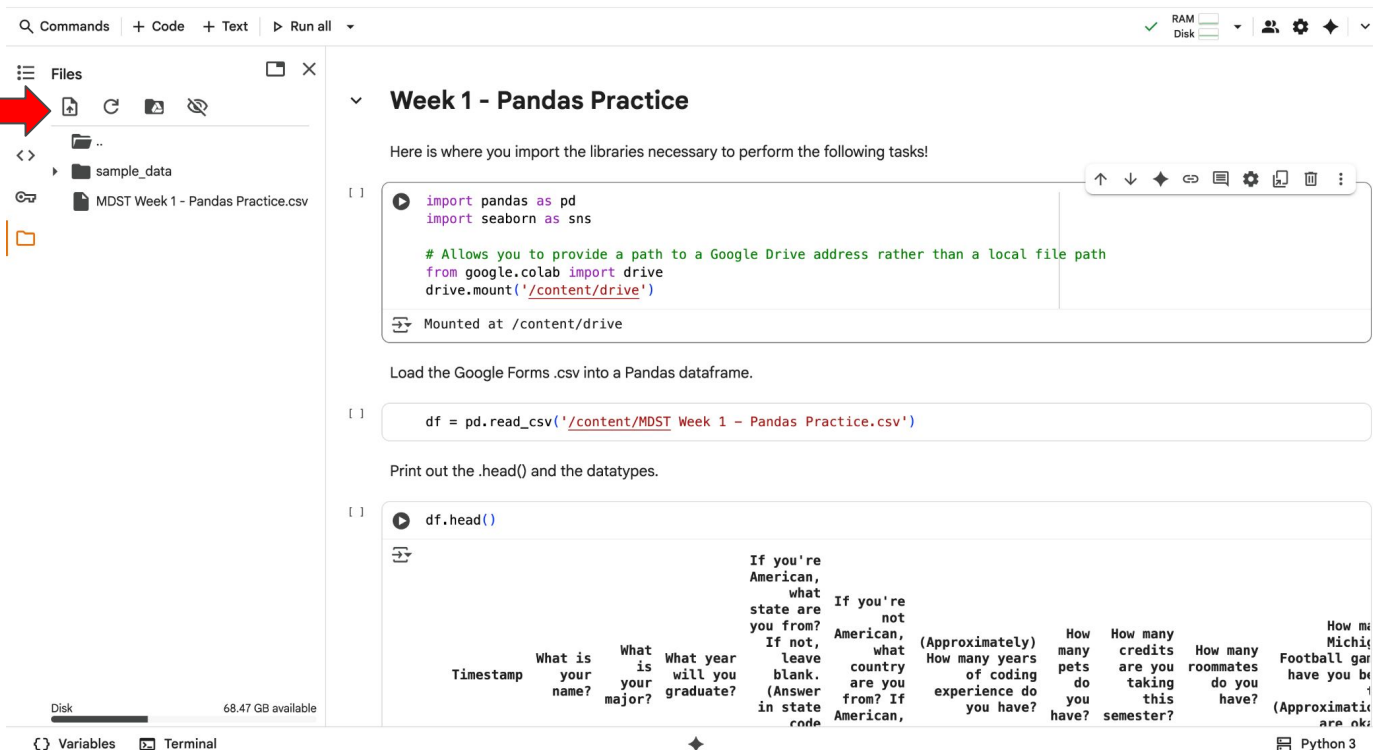
Below the code cell, the output of the .head() method is displayed as a table with 10 columns. The first column is 'Timestamp'. The other columns contain various survey questions. The output is truncated on the right side.

Timestamp	What is your name?	What is your major?	What year will you graduate?	If not, leave blank. (Answer in state code)	If you're American, what state are you from?	If you're not American, what country are you from? If American,	(Approximately) How many years of coding experience do you have?	How many pets do you have?	How many credits are you taking this semester?	How many roommates do you have?	How many Football games have you been to?

At the bottom of the notebook, there are tabs for 'Variables' and 'Terminal', and a status bar indicating 'Python 3'.

How to Upload Files into Colab

Click the upload button and select the file you want to upload



The screenshot shows the Google Colab interface. On the left, the 'Files' pane displays a directory structure with a folder named 'sample_data' and a file named 'MDST Week 1 - Pandas Practice.csv'. A red arrow points to the upload button (a square with a plus sign) in the top left of the Files pane. The main area is titled 'Week 1 - Pandas Practice' and contains the following text: 'Here is where you import the libraries necessary to perform the following tasks!'. Below this, a code cell contains the following Python code:

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

The output of the code cell shows 'Mounted at /content/drive'. Below the code cell, the text 'Load the Google Forms .csv into a Pandas dataframe.' is followed by a code cell containing:

```
df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

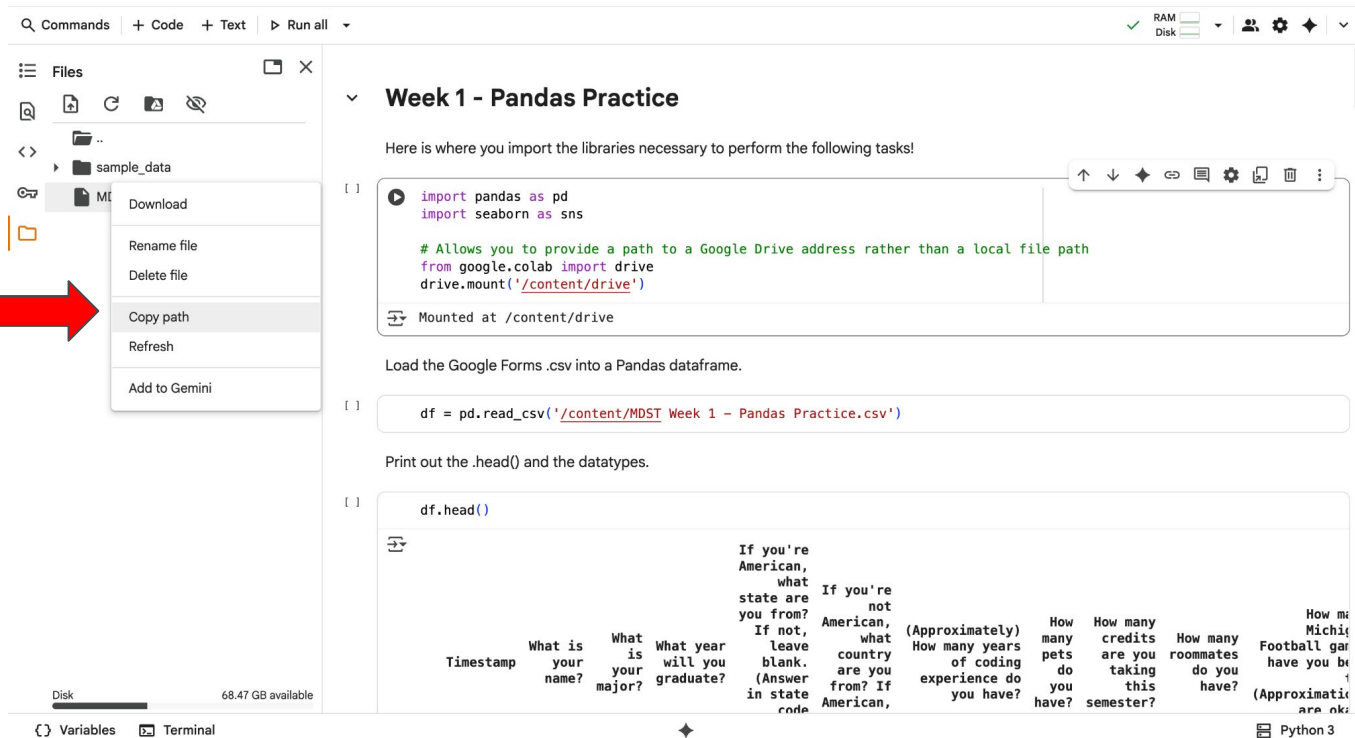
Below this, the text 'Print out the .head() and the datatypes.' is followed by a code cell containing:

```
df.head()
```

The output of the code cell shows the first five rows of the dataset, which includes columns for 'Timestamp', 'What is your name?', 'What is your major?', 'What year will you graduate?', 'If you're American, what state are you from?', 'If not, what country are you from?', 'If you're American, what state are you from?', 'If not, what country are you from?', 'How many years of coding experience do you have?', 'How many pets do you have?', 'How many credits are you taking this semester?', 'How many roommates do you have?', 'How many football games have you been to?', and 'How many Michigan are you from?'. The output is displayed in a table format.

How to Upload Files into Colab

Click the three dots and copy the path. Put this in your read function



The screenshot shows the Google Colab interface. On the left, the 'Files' pane displays a folder named 'sample_data' containing a file 'MDST Week 1 - Pandas Practice.csv'. A context menu is open for this file, with the 'Copy path' option highlighted. A red arrow points from the text 'Click the three dots and copy the path. Put this in your read function' to this option. The main workspace is titled 'Week 1 - Pandas Practice' and contains the following text and code:

Here is where you import the libraries necessary to perform the following tasks!

```
[ ] import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Load the Google Forms .csv into a Pandas dataframe.

```
[ ] df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

Print out the .head() and the datatypes.

```
[ ] df.head()
```

The output of `df.head()` is a table with columns: Timestamp, What is your name?, What is your major?, What year will you graduate?, If you're American, what state are you from?, If not, leave blank. (Answer in state code), If you're American, what country are you from? If American, (Approximately) How many years of coding experience do you have?, How many pets do you have?, How many credits are you taking this semester?, How many roommates do you have?, How many Football games have you been to? (Approximately) How many are ok.

At the bottom, there are tabs for 'Variables' and 'Terminal', and a 'Python 3' label.

How to Mount Your Google Drive

```
from google.colab import drive

drive.mount('/content/drive')

pd.read_csv('/content/drive/MyDrive/[FILE NAME]')
```

****NOTE:** If you saved your dataset in a folder (not just loose in your MyDrive folder), you will need to add further code to the file path before the file name in the final line. For example, if I saved my data called “alzheimers.csv” in a folder called “MDST-W26,” my read_csv statement would read:

```
pd.read_csv('/content/drive/MyDrive/MDST-W26/alzheimers.csv')
```